Manual Técnico de Cookly

Plataforma web de recetas colaborativas · Documentación técnica del proyecto

Basado en el análisis del repositorio cookly (rama santiago, último commit del 29/10/2026).

Tabla de contenido

1. Descripción general....2
2. Stack tecnológico....2
3. Arquitectura....2
4. Estructura del proyecto....3
5. Instalación y puesta en marcha....3
6. Modelo de datos....4
7. Configuración y capa de acceso a datos....4
8. Autenticación, sesiones y roles (includes/auth.php)....4
9. Capa de lógica de negocio....5
    9.1 funciones_usuarios.php....5
    9.2 funciones_recetas.php....5
    9.3 Proceso de guardar_receta....5
10. Módulos y páginas....6
    10.1 Flujo de moderación....6
    10.2 Estadísticas del panel....7
11. Frontend....7
12. Endpoint AJAX....7
13. Seguridad: lo que está bien y lo que debe corregirse....7
14. Problemas de portabilidad y mantenimiento....8
15. Control de versiones....9
16. Recomendaciones de mejora futura....9
17. Glosario....9

1. Descripción general

Cookly es una aplicación web para registrar, compartir y explorar recetas de cocina. Los usuarios se registran, publican recetas con ingredientes e imagen, y exploran las recetas de la comunidad con búsqueda en tiempo real. Las recetas nuevas o editadas pasan por un flujo de moderación: un administrador las aprueba o rechaza antes de que sean visibles públicamente. El administrador además dispone de un panel con estadísticas, gráficas y exportación a Excel/PDF.

Funcionalidades principales

- Registro, inicio y cierre de sesión con contraseñas cifradas.
- Gestión de perfil: datos, avatar, biografía, cambio de contraseña y eliminación de cuenta.
- CRUD de recetas propias (crear, ver, editar, eliminar) con ingredientes dinámicos y carga de imagen.
- Exploración paginada de recetas aprobadas y buscador AJAX por título, categoría o autor.
- Moderación de recetas (Pendiente / Aceptado / Rechazado) por el administrador.
- Panel de administración con KPIs, gráficas (Google Charts), exportación a Excel e impresión a PDF.
- Manual de usuario y página de video de presentación integrados.

2. Stack tecnológico

Capa	Tecnología
Lenguaje backend	PHP (sin framework, páginas procedurales)
Base de datos	MySQL / MariaDB, acceso mediante PDO
Frontend	HTML5, CSS3 propio (tema "Modern Organic"), JavaScript vanilla
Tipografía	Google Fonts – Nunito
Gráficas	Google Charts (gstatic.com/charts/loader.js)
Entorno de desarrollo esperado	XAMPP / WAMP (Apache + MySQL, usuario root sin contraseña)
Control de versiones	Git (ramas: main, santiago, castro, alison, pabon, piernagorda)

No existen `composer.json`, `package.json` ni dependencias externas instaladas: el proyecto funciona copiándolo al directorio público del servidor web.

3. Arquitectura

La aplicación sigue un patrón de **páginas PHP independientes (page controller)** que comparten una capa de includes con funciones de negocio y acceso a datos. No hay un enrutador central: cada URL corresponde a un archivo `.php`.

```
Navegador
   |  HTTP (GET/POST, fetch AJAX)
```

```
   v
Páginas PHP (index, recetas, receta_*, perfil, admin, ...)
   |  require_once
   v
includes/auth.php ........... sesión, roles, mensajes flash
includes/funciones_usuarios.php ... lógica de usuarios
includes/funciones_recetas.php .... lógica de recetas y estadísticas
   |  usan $pdo (global)
   v
config/database.php ......... conexión PDO
   v
MySQL (base de datos cookly)
```

Cada página sigue el mismo ciclo: (1) cargar `auth.php` y las funciones, (2) exigir login/rol, (3) procesar el POST si existe, (4) consultar datos, (5) renderizar con `header.php` y `footer.php`.

4. Estructura del proyecto

```
cookly/
+-- config/database.php          Conexión PDO
+-- includes/
|   +-- auth.php                 Sesión, roles, flash messages
|   +-- funciones_usuarios.php   CRUD de usuarios
|   +-- funciones_recetas.php    CRUD de recetas, búsqueda, estadísticas, moderación
|   +-- header.php               Cabecera HTML + navegación
|   +-- footer.php               Pie + carga de main.js
+-- public/
|   +-- css/  main.css, auth.css
|   +-- JS/   main.js, auth.js
|   +-- IMG/  logo, capturas del manual, avatares/, recetas/
+-- index.php                    "Mis recetas"
+-- recetas.php                  Explorar (paginado + buscador)
+-- ajax_buscar.php              Endpoint JSON de búsqueda
+-- receta_crear.php / receta_editar.php / receta_detalle.php / receta_eliminar.php
+-- perfil.php
+-- Login.php / Logout.php / registro.php
+-- admin.php                    Panel de administración
+-- exportar_excel.php           Reporte .xls
+-- reportes.php                 Estadísticas para usuarios logueados
+-- manual.php                   Manual de usuario
+-- video.php                    Video de presentación
+-- 404.php
```

5. Instalación y puesta en marcha

1. Instalar un servidor local con Apache, PHP 8.x y MySQL (por ejemplo XAMPP).
2. Copiar la carpeta `cookly` en el directorio público (`htdocs` o `www`).
3. Crear la base de datos `cookly` con codificación `utf8mb4` y las tablas descritas en la sección 6. **El repositorio no incluye un script `.sql`**, por lo que debe reconstruirse a partir de ese esquema o solicitarlo al equipo.
4. Revisar las credenciales en `config/database.php` (por defecto: host `localhost`, usuario `root`, contraseña vacía).
5. Dar permisos de escritura a `public/IMG/recetas/` y `public/IMG/avatares/` (se crean automáticamente si no existen y los permisos del padre lo permiten).

6. Abrir `http://localhost/cookly/login.php`, registrar un usuario y, para crear un administrador, cambiar manualmente su columna `rol` a `admin` en la tabla `Usuarios`.
7. Si se desea el video de presentación, colocar `public/video_presentacion.mp4` (el archivo no está en el repositorio).

6. Modelo de datos

El esquema no está versionado; la siguiente reconstrucción se deduce de las consultas SQL del código.

Usuarios: `id_usuario` (PK), `nombre`, `correo_electronico`, `contrasena_hash`, `avatar_url`, `biografia`, `rol` (`usuario` | `admin`).

Categorias: `id_categoria` (PK), `nombre`.

Recetas: `id_receta` (PK), `id_categoria` (FK), `id_usuario` (FK), `titulo`, `descripcion`, `instrucciones`, `tiempo_preparacion`, `dificultad` (Fácil / Media / Difícil), `porciones`, `estado` (Pendiente / Aceptado / Rechazado), `imagen_url`.

Ingredientes: `id_ingrediente` (PK), `nombre`.

Receta_Ingredientes: `id_receta` (FK), `id_ingrediente` (FK), `cantidad`, `unidad_medida`. Relación muchos a muchos entre recetas e ingredientes.

Relaciones: un usuario tiene muchas recetas; una categoría tiene muchas recetas; una receta tiene muchos ingredientes a través de `Receta_Ingredientes`. La pantalla de perfil indica que al eliminar un usuario se borran sus recetas, por lo que se asume `ON DELETE CASCADE` en las claves foráneas.

7. Configuración y capa de acceso a datos

`config/database.php` crea el objeto global `$pdo` con `utf8mb4`, modo de errores por excepciones (`ERRMODE_EXCEPTION`), `FETCH_ASSOC` por defecto y `EMULATE_PREPARES = false`, lo que garantiza sentencias preparadas reales. Todas las funciones de `includes/` acceden a `$pdo` mediante `global`.

8. Autenticación, sesiones y roles (includes/auth.php)

Función	Propósito
`estaAutenticado()`	Verifica que exista `$_SESSION['id_usuario']`
`requerirLogin()`	Redirige a `login.php` si no hay sesión
`esAdmin()` / `requerirAdmin()`	Verifica el rol `admin`; si no, redirige a `index.php`
`usuarioActual()`	Devuelve id, nombre, correo y rol de la sesión
`cerrarSesion()`	Vacía `$_SESSION`, expira la cookie y destruye la sesión
`set_flash_message()` / `mostrar_flash_message()`	Mensaje de un solo uso (tipos `exito` / `error`) mostrado por `header.php`

Flujo de login: `Login.php` recibe `correo` y `clave`, llama a `verificar_credenciales()` (busca el usuario y valida con `password_verify`) y guarda en sesión `id_usuario`, `nombre`, `correo_electronico` y `rol`. Las contraseñas se almacenan con `password_hash(..., PASSWORD_BCRYPT)`.

Registro: valida nombre (mínimo 3 caracteres), correo con `FILTER_VALIDATE_EMAIL`, contraseña de al menos 8 caracteres y coincidencia con la confirmación; rechaza correos duplicados.

9. Capa de lógica de negocio

9.1 funciones_usuarios.php

`crear_usuario`, `obtener_usuario_por_correo`, `obtener_usuario_por_id`, `verificar_credenciales`, `actualizar_perfil`, `actualizar_password`, `eliminar_usuario`.

9.2 funciones_recetas.php

Función	Descripción
`listar_categorias()`	Todas las categorías ordenadas por nombre
`listar_mis_recetas($id)`	Recetas del usuario, cualquier estado
`listar_todas_las_recetas($limite,$offset)`	Recetas **Aceptadas** con paginación
`contar_todas_las_recetas()`	Total de recetas aceptadas
`buscar_recetas_ajax($termino)`	Búsqueda `LIKE` por título, categoría o autor (máx. 20, solo aceptadas)
`obtener_receta($id)` / `obtener_ingredientes_receta($id)`	Detalle e ingredientes
`buscar_o_crear_ingrediente($nombre)`	Reutiliza o inserta un ingrediente en el catálogo
`receta_pertenece_a_usuario()`	Verificación de propiedad (definida, poco usada)
`guardar_receta($datos,$ings,$id=null)`	Crea o actualiza una receta (ver 9.3)
`eliminar_receta($id)`	Borra la receta
`obtener_estadisticas_generales()`	KPIs y agregaciones para reportes
`listar_recetas_pendientes()` / `cambiar_estado_receta()`	Moderación

9.3 Proceso de guardar_receta

1. Valida que tiempo y porciones sean mayores que 0 (si no, `die()` con un mensaje HTML).

2. Determina la imagen: si se sube un archivo con extensión `jpg/jpeg/png/webp/gif` se guarda en `public/IMG/recetas/` con nombre `uniqid('receta_')`; si no, se usa la URL indicada.
3. Convierte cada ingrediente del formulario en un `id_ingrediente`.
4. Abre una **transacción**: inserta (estado `Pendiente`) o actualiza la receta (también la devuelve a `Pendiente`), borra y reinserta sus filas en `Receta_Ingredientes`, y confirma con `commit`; ante error hace `rollBack`.

10. Módulos y páginas

Página	Acceso	Función
`Login.php`, `registro.php`	Público	Autenticación y alta de usuarios
`Logout.php`	Sesión	Cierra sesión
`index.php`	Usuario	Lista las recetas propias con su estado y acciones Ver/Editar/Eliminar
`recetas.php`	Usuario	Explorar recetas aceptadas, 6 por página, con buscador
`ajax_buscar.php`	Usuario	Devuelve JSON con tarjetas de resultados
`receta_crear.php`	Usuario	Formulario de nueva receta (queda Pendiente)
`receta_editar.php`	Dueño	Edita la receta y la devuelve a Pendiente
`receta_detalle.php`	Usuario	Detalle; si no está aceptada solo la ven el dueño y el admin
`receta_eliminar.php`	Dueño	Elimina por GET `?id=` y redirige a `index.php`
`perfil.php`	Usuario	Perfil, contraseña y eliminación de cuenta (confirmación escribiendo ELIMINAR)
`admin.php`	Admin	Moderación, KPIs, gráficas y descarga PDF/Excel
`exportar_excel.php`	Admin	Genera Reporte_Cookly_AAAA-MM-DD.xls (HTML con cabeceras de Excel)
`reportes.php`	Usuario	Versión de solo lectura de las estadísticas
`manual.php`, `video.php`, `404.php`	Usuario / público	Ayuda, video y página de error

10.1 Flujo de moderación

```
Crear receta --> Pendiente --> Admin aprueba  --> Aceptado (visible en Explorar)
                  ^    \-> Admin rechaza  --> Rechazado (solo visible al autor)
                  \-- Editar receta (vuelve a Pendiente)
```

10.2 Estadísticas del panel

`obtener_estadisticas_generales()` devuelve totales de usuarios, recetas y categorías, recetas por categoría, recetas por dificultad y el top 3 de usuarios con más recetas. El panel las dibuja con Google Charts (pastel y barras) y las repite en tablas. El PDF se obtiene con `window.print()` y una hoja `@media print` que oculta la navegación y los botones.

11. Frontend

- **header.php** calcula `$base_url` a partir de `SCRIPT_NAME` para que los enlaces funcionen desde cualquier carpeta, marca la pestaña activa y muestra el enlace Admin solo a administradores.
- **main.js** implementa: confirmación de eliminación (`data-confirm`), filas dinámicas de ingredientes (agregar/quitar), animación escalonada de tarjetas, estado de carga en formularios con archivos y el buscador en tiempo real con debounce de 300 ms que restaura el HTML original al vaciar el campo.
- **auth.js**: mostrar/ocultar contraseña y validación de coincidencia en el registro.
- **main.css** define variables CSS (paleta verde esmeralda `#0F766E`, fondo `#F3F4F6`, radios, sombras y transiciones) y los componentes: tarjetas, grid, formularios y barra superior fija. **auth.css** estiliza login y registro.

12. Endpoint AJAX

`GET ajax_buscar.php?q=<texto>` — requiere sesión. Respuesta `application/json`:

```
{ "html": "<article class=\"recipe-card\">...</article>" }
```

Si `q` está vacío devuelve `html` vacío; si no hay coincidencias devuelve un bloque "No se encontraron recetas".

13. Seguridad: lo que está bien y lo que debe corregirse

Buenas prácticas ya aplicadas

- Sentencias preparadas con PDO en todas las consultas.
- Hash BCrypt con `password_hash` / `password_verify`.
- Escapado de salida con `htmlspecialchars` en la gran mayoría de vistas.
- Control de acceso por rol en `admin.php` y `exportar_excel.php`, y control de propiedad en editar y eliminar.
- Visibilidad restringida de recetas no aprobadas.

Riesgos detectados y recomendaciones

#	Hallazgo	Recomendación
1	Eliminar receta se ejecuta por GET sin token: vulnerable a CSRF (un enlace o imagen externa puede borrar recetas).	Usar POST con token CSRF.

#	Hallazgo	Recomendación
2	No hay tokens CSRF en ningún formulario (perfil, eliminar cuenta, moderación).	Generar un token por sesión y validarlo.
3	No se llama a `session_regenerate_id(true)` tras el login (fijación de sesión).	Regenerar el id al autenticar.
4	Subida de archivos validada solo por extensión.	Validar MIME real (`finfo`), tamaño máximo y regenerar la imagen.
5	Credenciales root sin contraseña en `config/database.php`, versionadas en Git.	Usar un usuario con privilegios mínimos y variables de entorno / archivo fuera del repositorio.
6	Errores de MySQL y de conexión se muestran al usuario (`die($e->getMessage())`).	Registrar en log y mostrar un mensaje genérico.
7	Valores sin escapar en vistas: nombre de categoría en `receta_crear.php`, `estado` y `cantidad/unidad` en `receta_detalle.php`.	Aplicar `htmlspecialchars` en todas las salidas.
8	`perfil.php` no valida que el correo nuevo sea único ni su formato en servidor.	Validar y capturar la excepción de duplicados.
9	Al cambiar de correo/nombre no se valida longitud; no se limitan intentos de login.	Añadir validaciones y límite de intentos.
10	`reportes.php` es accesible para cualquier usuario logueado y expone estadísticas globales.	Restringir con `requerirAdmin()` si no es intencional.
11	La búsqueda `LIKE` no escapa `%` y `_` del término.	Escapar comodines.
12	En `admin.php` la receta puede reaprobarse sin revisar su estado actual.	Validar la transición de estados.

14. Problemas de portabilidad y mantenimiento

- **Mayúsculas/minúsculas en rutas:** los archivos se llaman `Login.php` y `Logout.php`, pero el código redirige a `login.php` y `logout.php`; además las vistas cargan `public/CSS/...` mientras la carpeta real es `public/css`. Funciona en Windows, pero **falla en servidores Linux**. Unificar a minúsculas.
- **Ruta relativa en el footer:** `footer.php` carga `public/JS/main.js` sin `$base_url`, lo que puede romper el script en páginas ubicadas en subdirectorios.
- **Falta el script SQL** del esquema y datos iniciales (categorías, administrador). Se recomienda añadir `database/cookly.sql` o migraciones.
- **Archivo ausente:** `public/video_presentacion.mp4` no existe en el repositorio.
- **Acoplamiento a `global $pdo`:** dificulta las pruebas; considerar inyección de dependencias o una clase de conexión.

- **Lógica duplicada:** el armado de tarjetas se repite en `index.php`, `recetas.php`, `admin.php` y `ajax_buscar.php`; la extracción de ingredientes se repite en crear y editar. Conviene extraerlos a funciones o plantillas parciales.
- **Uso de `die()` para errores de validación** dentro de la capa de negocio; es preferible devolver errores y mostrarlos en el formulario.
- **Estilos en línea** abundantes en las vistas; trasladarlos a `main.css`.
- **README vacío:** solo contiene el título del proyecto.
- **Exportación a Excel** generada como HTML con extensión `.xls`; Excel puede mostrar una advertencia al abrirlo. Considerar una librería como PhpSpreadsheet para un `.xlsx` real.
- **Validación de ingredientes:** no se comprueba que la cantidad sea numérica positiva en el servidor.

15. Control de versiones

El repositorio tiene aproximadamente 16 commits entre el 27 y el 29 de octubre de 2026, con ramas por integrante del equipo (`santiago`, `castro`, `alison`, `pabon`, `piernagorda`) integradas en `main` mediante pull requests. Hitos visibles en el historial: versión inicial del proyecto, generación de reportes en PDF y Excel, validaciones, buscador en tiempo real y corrección de rutas del logo.

16. Recomendaciones de mejora futura

1. Incorporar el script de base de datos y un README con instrucciones de instalación.
2. Implementar CSRF, regeneración de sesión y validación robusta de archivos.
3. Migrar la configuración sensible a variables de entorno.
4. Añadir gestión desde el panel (promover usuarios, administrar categorías) y poder eliminar recetas desde el panel de administración.
5. Mostrar el motivo del rechazo al autor de la receta.
6. Agregar recuperación de contraseña y verificación de correo.
7. Añadir favoritos, comentarios y valoraciones (la interfaz ya incluye la clase `.rating`).
8. Introducir un enrutador y plantillas (por ejemplo Composer + un micro-framework) si el proyecto crece.
9. Escribir pruebas automatizadas para las funciones de negocio.

17. Glosario

- **PDO:** capa de abstracción de PHP para acceder a bases de datos.
- **Flash message:** mensaje que se muestra una sola vez tras una redirección.
- **CSRF:** ataque que fuerza a un usuario autenticado a ejecutar acciones no deseadas.
- **Debounce:** técnica que retrasa una acción hasta que el usuario deja de escribir.
- **Moderación:** proceso de revisión de recetas por el administrador antes de publicarse.